Data Science Intern at Data Glacier

Project: Hate Speech Detection using Transformers (Deep Learning)

Week 13: Deliverable

Group Name: ToxiTrackers

Name: Mehedi Hasan Shakil

University: Chittagong University of Engineering & Technology

Email: shakilcuetcse@gmail.com

Country: Bangladesh

Specialization: Data Science

Batch Code: LISUM44

Date: 30 June 2025

Submitted to: Data Glacier

Table of Contents:

1. Project Plan

Weeks	Date	plan
Weeks 07	May 19, 2025	Problem Description, Business understanding, Data Intake Report
Weeks 08	May 26, 2025	Data Preprocessing
Weeks 09	June 2, 2025	Data Cleansing and Transformation
Weeks 10	June 9, 2025	EDA performed
Weeks 11	June 16, 2025	EDA Presentation and Proposed Modelling Technique
Weeks 12	June 23, 2025	Model Selection and Model Building
Weeks 13	June 30, 2025	Final Submission (Report + Code + Presentation)

2. Problem Description

Hate speech poses a significant challenge on digital platforms, often manifesting as verbal, written, or behavioral communication targeting individuals or groups based on religion, ethnicity, nationality, race, gender, or other identity traits. With the rise of social media usage, particularly platforms like Twitter, monitoring and mitigating such harmful content has become essential.

This project aims to build an automated hate speech detection system using advanced Natural Language Processing (NLP) techniques and transformer-based deep learning models. The task is framed as a binary text classification problem, where the model learns to distinguish between hateful and non-hateful tweets. The goal is to create a reliable and scalable model capable of supporting content moderation efforts by detecting hate speech in real time.

3. Business Understanding

Social media platforms are essential tools for global communication but also serve as mediums for the spread of hate speech and abusive content. The unchecked proliferation of such content can lead to social unrest, legal issues, and reputational damage for platform providers.

Businesses operating these platforms are increasingly under pressure to ensure user safety and compliance with regulations on digital conduct.

By deploying an automated hate speech detection system, companies can significantly enhance their moderation capabilities. This not only helps maintain a safer online environment but also improves user trust and brand integrity. From a data science perspective, this project addresses key business goals such as minimizing manual moderation efforts, reducing harmful interactions, and supporting ethical AI practices in content governance.

4. Data Preprocessing

In this section, we describe the text preprocessing techniques applied to prepare the dataset for machine learning models. Raw text data typically contains various inconsistencies, noise, and irrelevant components that can negatively affect the performance of NLP-based models. Therefore, a sequence of cleaning steps was carried out to normalize and refine the input tweets.

4.1 Text Cleaning

Text data collected from social media platforms such as Twitter often contains unwanted elements like user mentions, hyperlinks, emojis, and non-standard formatting. Cleaning such data is essential for ensuring better feature extraction and model performance. The steps below summarize our cleaning pipeline:

4.1.1 Lowercasing

All characters in the text were converted to lowercase. This normalization step ensures that words like *"Hate"* and *"hate"* are treated identically. Without this transformation, these words would be considered different features in the model, leading to unnecessary sparsity and dimensionality in the feature space.

4.1.2 Removing User Mentions

Twitter handles or user mentions (e.g., *@username*) were removed. These references are unique to individual users and do not contribute semantically to the classification task. Their removal helps avoid model distraction from non-informative tokens.

4.1.3 Removing URLs

URLs (web links) were stripped from the text. Since the task focuses on analyzing the sentiment or intent of the written content, web addresses—which usually lead to external content—do not provide meaningful context and are treated as noise.

4.1.4 Removing Special Characters and Digits

Special characters (e.g., `@`, `#`, `!`, `%`) and numeric digits were eliminated. These characters typically do not carry linguistic meaning in the context of sentiment or hate speech detection. A regular expression-based filter was applied to retain only alphabetic characters and necessary whitespace.

4.1.5 Stripping Whitespace

Any leading or trailing spaces in the tweets were removed. This ensures that the text is properly formatted and avoids inconsistencies during tokenization and further processing.

4.2 Remove Stopwords

Stopwords are common words such as “the”, “is”, “in”, “and”, etc., which do not carry significant meaning and are often removed to reduce noise in the text. In this step, the stopwords were removed from each tweet using the Natural Language Toolkit (NLTK)'s built-in English stopword list. This step helps to focus the analysis on the most relevant and informative words.

4.3 Tokenization

Tokenization involves breaking down each tweet into individual tokens or words. This process is essential for further natural language processing (NLP) tasks such as lemmatization and vectorization. Tokenization was performed using NLTK’s word tokenizer, which splits the cleaned text into word-level tokens while handling punctuation and whitespace effectively.

4.4 Lemmatization

Lemmatization is the process of reducing words to their base or dictionary form, known as a lemma. Unlike stemming, lemmatization considers the context of the word to ensure that it is grammatically correct. For instance, words like "running", "ran", and "runs" are reduced to the base form "run". This was performed using spaCy's lemmatizer, helping to normalize the text and reduce feature dimensionality.

5. Feature Extraction

After preprocessing the text data, several feature extraction techniques were employed to convert the textual data into numerical representations suitable for machine learning models. These

representations capture different aspects of the text and were used as input features for classification.

5.1 Bag of Words (BoW)

The Bag of Words model is a simple and commonly used text representation technique. It creates a vocabulary of all unique words in the dataset and represents each tweet as a vector containing the frequency of each word. The CountVectorizer from scikit-learn was used to transform the preprocessed tweets into sparse vectors. This method captures the presence and frequency of words but ignores grammar and word order.

5.2 Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF improves upon the Bag of Words model by not only considering the frequency of words but also their uniqueness across all documents. Words that appear frequently in one document but rarely across others are given higher weights. The TfidfVectorizer from scikit-learn was applied to the preprocessed text data to generate a TF-IDF matrix. This representation helps reduce the impact of common yet less informative words, making the feature space more meaningful for classification tasks.

5.3 Word Embeddings

To capture semantic relationships and contextual meanings, pre-trained word embeddings were used. Specifically, GloVe (Global Vectors for Word Representation) embeddings were loaded, and each tweet was represented as the average of its word vectors. This method enables the model to understand relationships between similar words (e.g., "king" and "queen") and provides dense, real-valued feature vectors that are more informative than BoW or TF-IDF.

5.4 BERT Embeddings

Bidirectional Encoder Representations from Transformers (BERT) is a deep learning model developed by Google that provides context-aware word embeddings. The bert-base-uncased pre-trained model was used via the Hugging Face Transformers library to extract embeddings. Each tweet was tokenized using BertTokenizer, and the [CLS] token's embedding was extracted from BertModel to represent the entire tweet. These embeddings are highly contextual and provide rich information suitable for downstream classification tasks. The extracted features can also be fine-tuned later with BERT’s classification head for improved performance.

6. EDA (Exploratory Data Analysis)

In this part, we do a simple analysis to understand the data's structure and patterns. We check the number of total words in tweets and find the most frequent words appearing in hate and non-hate tweets. Then, we check the class distribution and use a resampling technique to solve the imbalance issue.

6.1 Number of Words per Tweet

We calculated the number of words in each tweet after the preprocessing steps. The distribution is visualized in *Figure 6.1*, which shows that the majority of tweets contain between 5 to 15 words. This indicates a relatively concise structure, which is typical for Twitter posts.

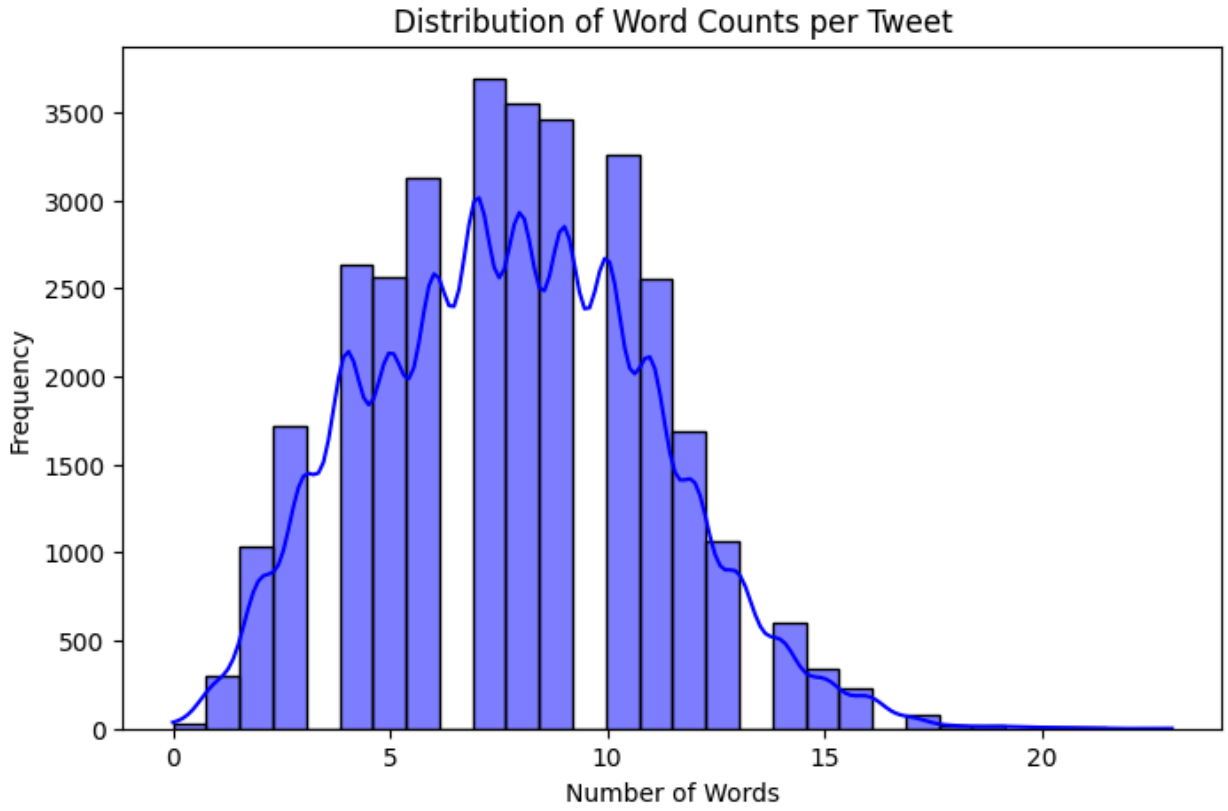

Figure 6.1: Histogram showing the distribution of word counts per tweet.

6.2 Most Frequent Words in the Entire Dataset

The 20 most frequently occurring words across the entire dataset are shown in Figure 6.2. These terms were extracted from the lemmatized token lists and represent the most commonly used language elements in both hate and non-hate tweets.

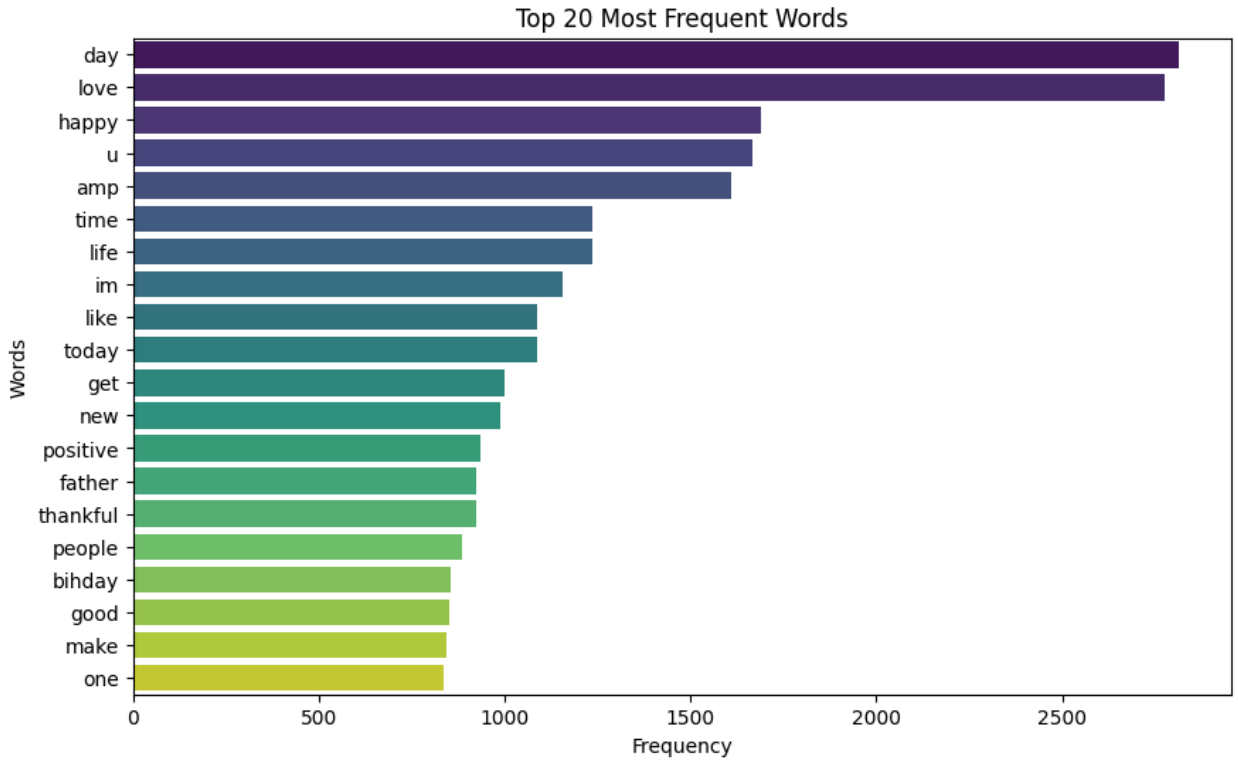

Figure 6.2: Bar plot of the top 20 most frequent words across all tweets.

6.3 Most Frequent Words in Hate vs Non-Hate Tweets

To distinguish the linguistic patterns between hate and non-hate tweets, we separately analyzed the most frequent words in each class. As shown in Figure 6.3, hate tweets tend to use more aggressive or targeted language, while non-hate tweets exhibit more neutral or conversational vocabulary.

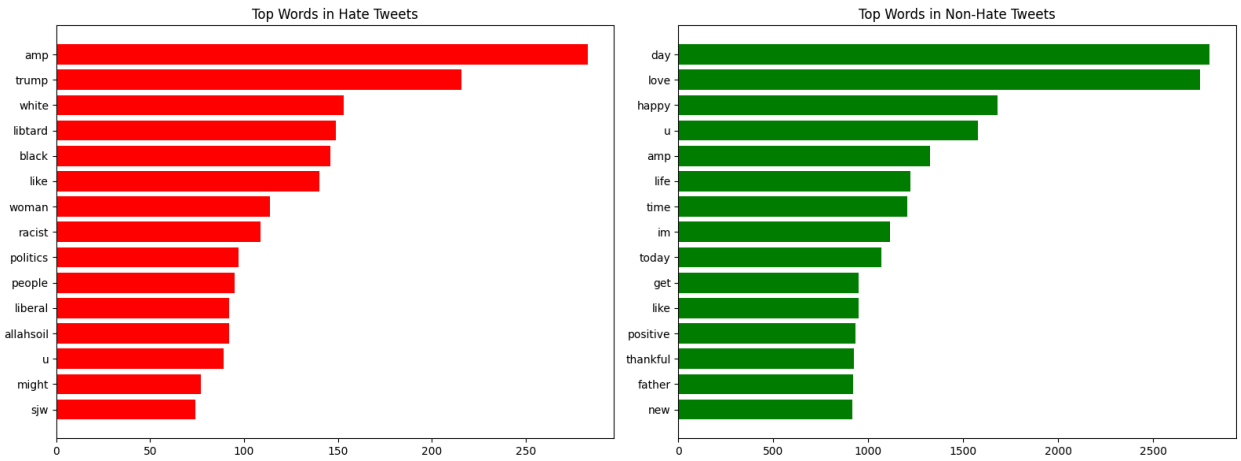

Figure 6.3: Horizontal bar plots of the top 15 frequent words in hate vs. non-hate tweets.

6.4 Word Cloud for Each Class

Word clouds offer a visually intuitive way to understand the most prominent words in each category. Figure 6.4 shows the word clouds for hate and non-hate tweets, respectively. The size of each word in the cloud corresponds to its frequency. Hate-related tweets reveal more emotionally charged or offensive language, whereas non-hate tweets reflect diverse, less targeted content.

Figure 6.4: Word cloud for hate speech tweets and non-hate speech tweets.

6.5 Hate vs. Non-Hate Tweet Distribution

The original class distribution is visualized in Figure 6.5. It is evident that the dataset is imbalanced, with a significantly higher number of non-hate tweets compared to hate tweets. This imbalance can affect the model's performance if not properly addressed.

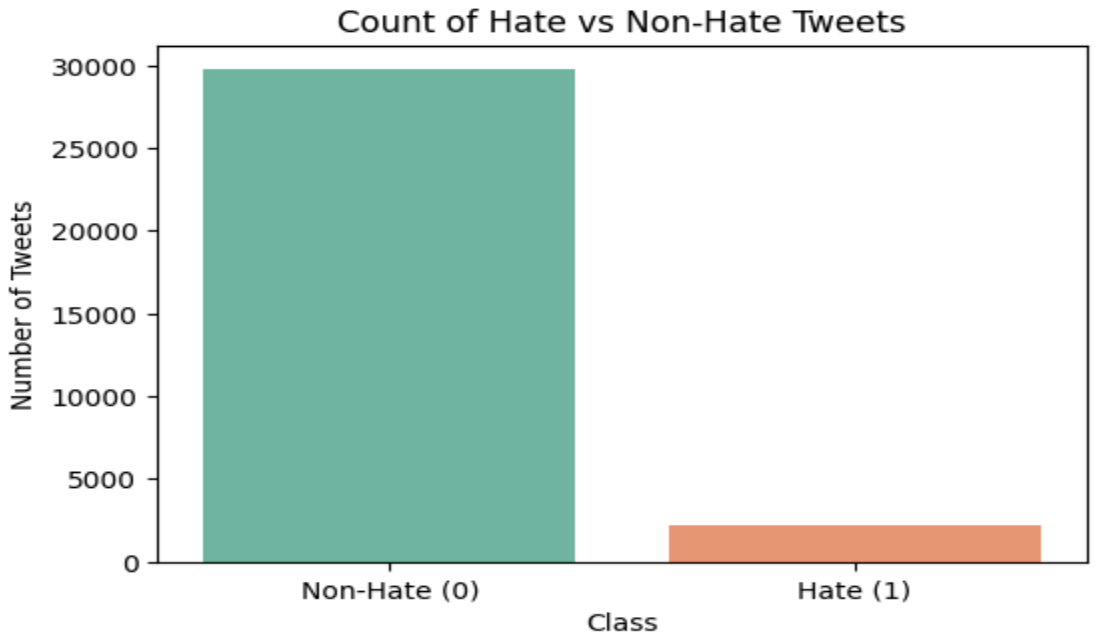

Figure 6.5: Count plot showing distribution of hate vs. non-hate tweets.

6.6 Handling Class Imbalance

To mitigate the impact of class imbalance, we applied upsampling to the minority class (hate tweets). After resampling, both classes have equal representation in the dataset, as shown in Figure 6.6. This step ensures that the model is trained on a balanced dataset, which improves its ability to generalize across both classes.

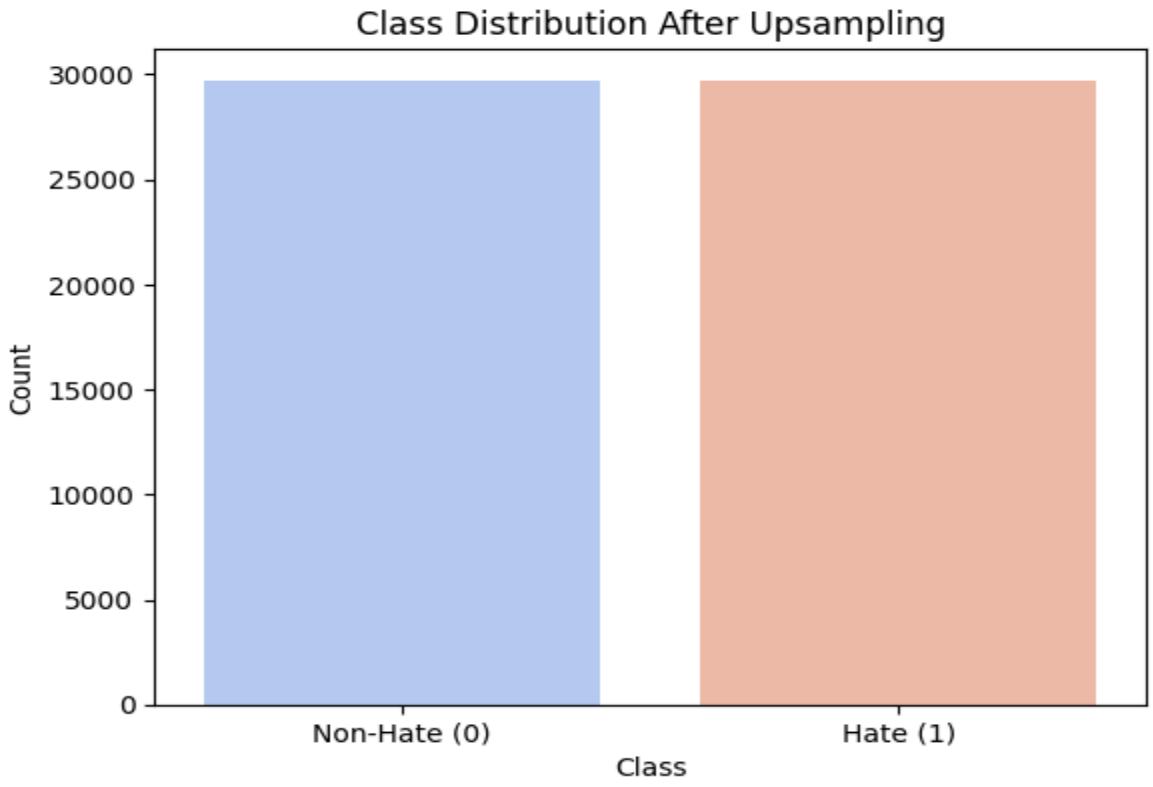

Figure 6.6: Class distribution after upsampling the minority class (hate tweets).

This EDA helped us better understand the nature of the dataset, uncover class imbalance, and prepare for effective model training in the subsequent steps.

7. Model Building and Training

After completing extensive preprocessing and balancing the dataset, this section focuses on developing and training various machine learning and deep learning models to perform binary classification for hate speech detection. Our approach is driven by both the diversity of model families and the suitability of feature extraction methods aligned with each model's strengths. The dataset was split into 80% training and 20% testing across all feature types to ensure

consistent evaluation. Three different types of features—TF-IDF, GloVe, and BERT embeddings—were used depending on the model architecture. TF-IDF, a sparse, high-dimensional representation, was used with traditional machine learning models. Dense, semantic embeddings such as GloVe and BERT were used with deep learning architectures to capture contextual and semantic information.

7.1 Support Vector Machine (SVM) using TF-IDF

Support Vector Machine is a powerful linear classifier that performs exceptionally well in high-dimensional spaces. Given that TF-IDF generates a sparse matrix capturing word frequency relevance, SVM was selected as a suitable candidate for this representation. We used the LinearSVC variant from scikit-learn due to its efficiency and effectiveness with sparse features. This model was trained on the TF-IDF features extracted from the balanced dataset. Its simplicity, strong theoretical foundation, and ability to generalize well make it an appropriate baseline linear model for the hate speech classification task.

7.2 Random Forest using TF-IDF

The Random Forest classifier was selected as an ensemble-based model that combines multiple decision trees to enhance prediction accuracy and reduce overfitting. TF-IDF features, though sparse, work effectively with Random Forest since the model is capable of handling high-dimensional input and learning complex non-linear patterns. In this experiment, the model was trained with 100 trees and a fixed random state to ensure reproducibility. The interpretability of feature importance and robustness of the algorithm make it a reliable choice in traditional machine learning pipelines for text classification tasks.

7.3 XGBoost using TF-IDF

XGBoost, or eXtreme Gradient Boosting, is known for its performance and scalability in classification problems. It builds additive models in a forward stage-wise fashion and uses a gradient descent algorithm to minimize errors. Given the suitability of TF-IDF features for traditional models, XGBoost was trained using these representations. The model is optimized with log-loss as the evaluation metric and performs well in handling imbalanced classes and sparse input, both of which are relevant to our problem domain. Its regularization capabilities and advanced tree pruning techniques help it generalize effectively.

7.4 CNN + LSTM using GloVe Embeddings

For deep learning, we implemented a hybrid model combining Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks. GloVe embeddings, which map words into dense vector spaces that reflect semantic similarity, were used as input. The CNN layer

captures local spatial patterns like n-grams in the input sequence, while the LSTM layer models sequential dependencies, which is essential for understanding the temporal flow of language. This architecture is particularly beneficial for detecting hate speech patterns that depend on both the presence and order of words. The model was trained over multiple epochs with dropout applied to reduce overfitting.

7.5 Transformer-Based Model using BERT Embeddings

Instead of fine-tuning the entire BERT architecture—which is computationally intensive—we extracted contextual BERT embeddings and trained a deep neural network as a classification head. BERT embeddings offer dense, rich, and context-aware representations of text, making them highly suitable for capturing nuanced language such as hate speech. The classification head consists of fully connected layers with ReLU activations and dropout for regularization, ending with a sigmoid activation to perform binary classification. This approach enables us to leverage BERT's contextual power while keeping the architecture lightweight and trainable on limited hardware resources.

8. Model Performance Evaluation

We evaluate our model performance from confusion matrix, precision, recall (sensitivity), F1-Score, ROC-AUC score, and ROC curve.

8.1 Confusion Matrix and Classification Report

To thoroughly evaluate the performance of the models used for hate speech detection, we employed standard classification metrics derived from the confusion matrix, including precision, recall (sensitivity), F1-score, and accuracy. These metrics provide a comprehensive understanding of how well each model distinguishes between hate and non-hate tweets.

A confusion matrix displays the counts of true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions. This allows us to visualize the types of errors made by a classifier and to compute other metrics accordingly:

- Precision = TP / (TP + FP): measures the model’s accuracy in predicting hate tweets.
- Recall = TP / (TP + FN): reflects the model’s ability to detect all hate tweets.
- F1-Score: the harmonic mean of precision and recall, providing a balanced measure.
- Accuracy = (TP + TN) / (TP + TN + FP + FN): the overall correctness of the model.

We evaluated five different models on the test set:

1. Support Vector Machine (SVM)
2. Random Forest (RF)
3. XGBoost
4. CNN + LSTM (using GloVe embeddings)
5. BERT

Below are the confusion matrices of these models, which illustrate their prediction performance visually:

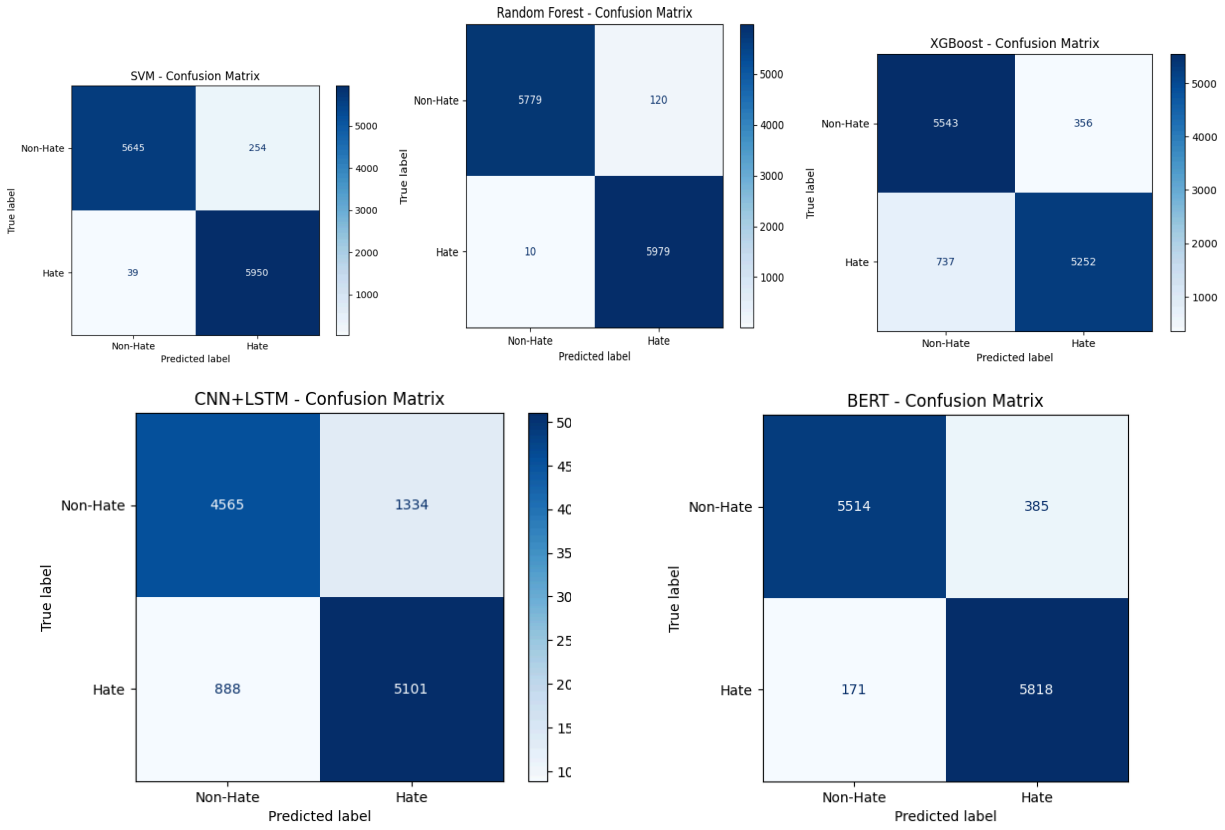

Figure 8.1: Confusion Matrices of 5 Models

These visualizations aid in understanding how frequently each model misclassifies hate and non-hate tweets. Among them, the Random Forest and BERT-based models show superior balance between precision and recall, making them more effective for practical deployment in hate speech detection scenarios. Classification reports of five model are given below:

```
Classification Report for SVM:

              precision    recall  f1-score   support

    Non-Hate       0.99      0.96      0.97      5899
        Hate       0.96      0.99      0.98      5989

    accuracy                           0.98     11888
   macro avg       0.98      0.98      0.98     11888
weighted avg       0.98      0.98      0.98     11888


Classification Report for Random Forest:

              precision    recall  f1-score   support

    Non-Hate       1.00      0.98      0.99      5899
        Hate       0.98      1.00      0.99      5989

    accuracy                           0.99     11888
   macro avg       0.99      0.99      0.99     11888
weighted avg       0.99      0.99      0.99     11888
```

```
Classification Report for XGBoost:

              precision    recall  f1-score   support

    Non-Hate       0.88      0.94      0.91      5899
        Hate       0.94      0.88      0.91      5989

    accuracy                           0.91     11888
   macro avg       0.91      0.91      0.91     11888
weighted avg       0.91      0.91      0.91     11888

372/372 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step

Classification Report for CNN+LSTM:

              precision    recall  f1-score   support

    Non-Hate       0.84      0.77      0.80      5899
        Hate       0.79      0.85      0.82      5989

    accuracy                           0.81     11888
   macro avg       0.81      0.81      0.81     11888
weighted avg       0.81      0.81      0.81     11888
```

```
372/372 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

Classification Report for BERT:

              precision    recall  f1-score   support

    Non-Hate       0.97      0.93      0.95      5899
        Hate       0.94      0.97      0.95      5989

    accuracy                           0.95     11888
   macro avg       0.95      0.95      0.95     11888
weighted avg       0.95      0.95      0.95     11888
```

Figure 8.2: Classification Reports of 5 Models

8.2 ROC-AUC Score

A performance statistic for assessing a binary classification model's quality is the ROC-AUC score. AUC is an acronym for Area Under the Curve, and ROC is an acronym for Receiver Operating Characteristic. It sheds light on how well the model can differentiate between positive and negative classes at various threshold settings. Better overall performance and discrimination abilities of the model are indicated by a higher ROC-AUC score.

Out of the five models we used, the Random Forest model has the best overall performance and discrimination abilities, as shown by its highest ROC-AUC score in the ROC-AUC score table in Figure 8.3.

```
SVM ROC-AUC Score: 0.9752
Random Forest ROC-AUC Score: 0.9890
XGBoost ROC-AUC Score: 0.9083
372/372 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step
CNN+LSTM ROC-AUC Score: 0.8930
372/372 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
BERT ROC-AUC Score: 0.9850
```

Figure 8.3: ROC-AUC Score of 5 Models

8.3 ROC Curve

A graphical depiction known as the ROC curve illustrates a binary classifier system's diagnostic performance when its discrimination threshold is changed; each point on the curve represents a distinct classification threshold. At different threshold settings, it plots the false positive rate (1 - specificity: the percentage of actual non-hate tweets that the model incorrectly classifies as hate tweets) against the true positive rate (Recall: the percentage of actual hate tweets that are correctly identified by the model).

The area under this curve, or AUC score, offers a comprehensive assessment of performance across all potential classification criteria. AUC scores vary from 0 to 1, with 1 denoting perfect classification and 0.5 denoting performance that is no better than random guessing.

From Figure 8.4, the Random Forest model has the highest AUC (area under the curve). It is the orange line, which is overlapped with the purple line and blue line. This model has the best ability to distinguish between hate and non-hate tweets across different threshold settings.

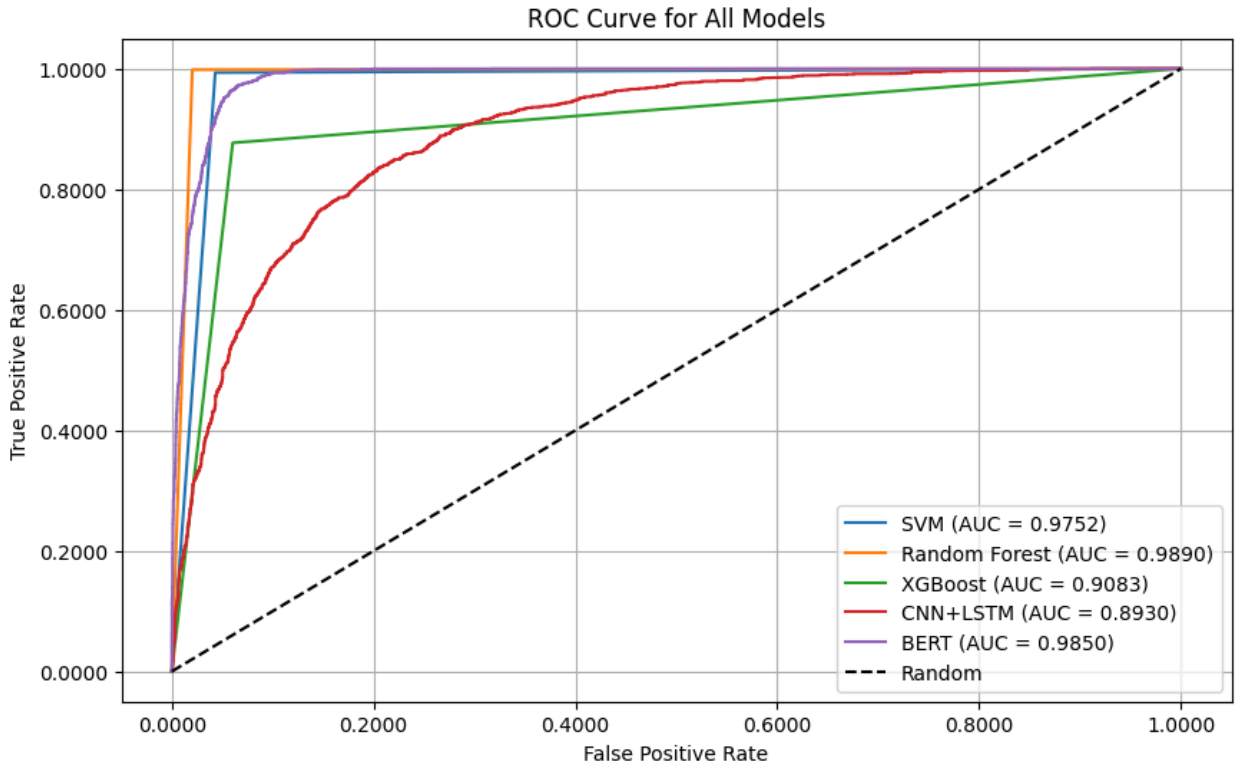

Figure 8.4: ROC Curve of 5 Models

9. Application Design

To demonstrate the real-world usability of the proposed hate speech detection model, we designed and deployed a web application using the Flask framework. The system allows users to input any tweet-like text and receive an immediate prediction indicating whether the content is hate speech or not. This section explains the system architecture, folder structure, deployment environment, and how the end-user interacts with the application.

9.1 Deploying ML Model on Flask to Create a Web Application

The deployment pipeline of our model is built with **Flask**, a lightweight Python web framework, chosen for its simplicity and integration capabilities. The architecture is divided into three major parts:

1. **Model Integration**: The trained Random Forest model and TF-IDF vectorizer are loaded into memory.
2. **Frontend (HTML/CSS)**: Provides a user-friendly interface for submitting tweets.
3. **Backend (Flask Server)**: Handles user input, preprocesses text, transforms it using the vectorizer, and feeds it to the classifier for prediction.

The overall workflow of how the web application operates is illustrated in Figure 9.1. When a user inputs a tweet, it is processed by the Flask backend, passed through the vectorizer and classifier, and finally the prediction is rendered back onto the web page.

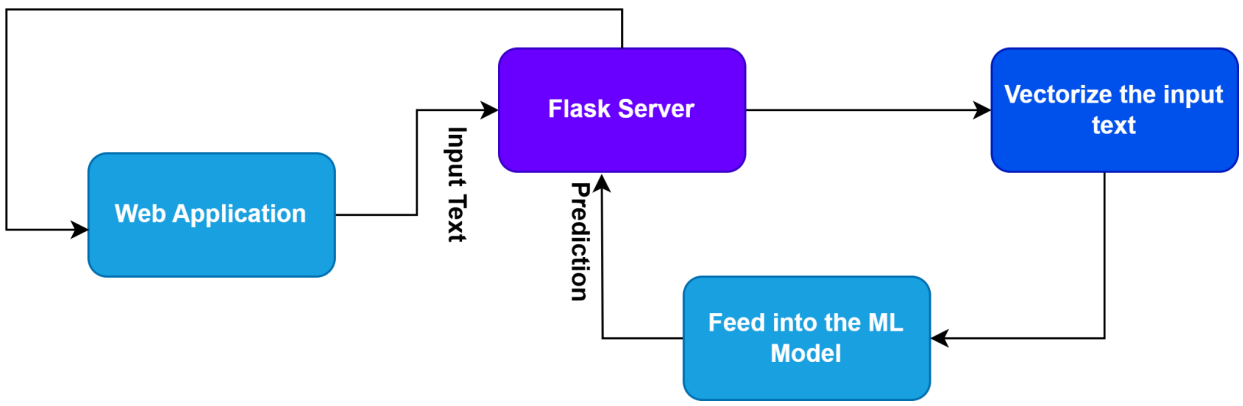

Figure 9.1: Web Application Workflow

In terms of file organization, the application follows a structured and modular directory layout as shown in Figure 9.2. This organization ensures easy scalability, maintenance,and clarity in development.

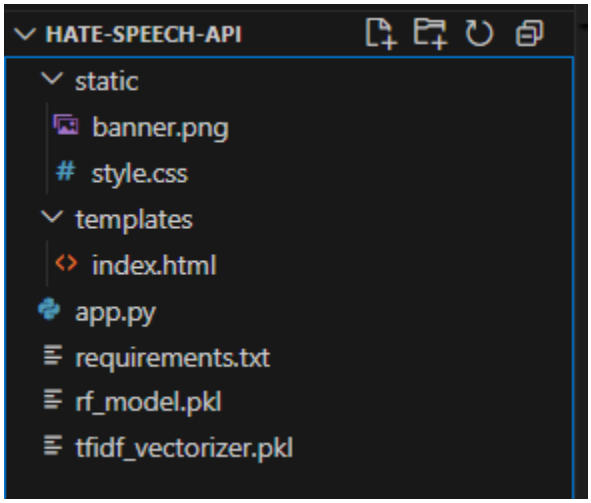

Figure 9.2 Application Folder Structure and Files

9.2 Running Procedure

Once deployed (either locally or on a cloud platform like Render), users can interact with the web application through a clean and modern web interface. The user simply pastes a tweet into the provided input box and clicks the “Analyze Tweet” button. The system then instantly returns a label indicating whether the content is considered Hate or Non-Hate speech.

To visually demonstrate the interface and output, we provide two sample screenshots:

Figure 9.3: Web application interface showing the detection of a hate speech tweet

Figure 9.4: Web application interface showing the detection of a non-hate speech tweet

These interfaces confirm the effective integration of the machine learning model into a user-accessible web environment, supporting our goal of practical and real-time hate speech detection.

10. Conclusion

This project successfully developed an automated hate speech detection system using a combination of machine learning and deep learning techniques. After thorough data preprocessing and feature extraction, five models were trained and evaluated—Random Forest and BERT-based models performed the best across key metrics. The final model was deployed through a Flask-based web application, enabling real-time tweet classification. Overall, this system demonstrates a practical and scalable solution to support content moderation on social media platforms.

Reference

[1] Dataset: https://www.kaggle.com/datasets/vkrahul/twitter-hate-speech/code

[2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780. DOI: 10.1162/neco.1997.9.8.1735

[3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.